

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа

Выполнила:

Шадрина Александра

Группа К3342, поток ФРЭНД 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2024 г.

Задача

Варианты остаются прежними. Теперь Вам нужно привязать то, что Вы делали в ЛР1 к внешнему API средствами fetch/axios/xhr. Реализуйте моковое API средствами JSON-сервера и подключите к нему авторизацию, как в примерах, которые мы рассматривали в рамках тем "Имитация работы с API".

Вариант – Сервис для обмена рецептами и кулинарных блогов.

- Вход
- Регистрация
- Личный кабинет пользователя (сохраненные рецепты, публикации)
- Поиск рецептов с фильтрацией по типу блюда, сложности, ингредиентам
- Страница рецепта с фото, пошаговыми инструкциями и видео
- Социальные функции (комментарии, лайки, подписки на кулинаров)

Ход работы

Для начала скачиваем с официального сайта Node.js. Далее необходимо установить JSON-сервер в консоли проекта с помощью команды `npm install json-server`. Почему-то в таком случае он не виден, пробуем установить глобально: `npm install -g json-server`. Далее проверяем установку:

```
PS C:\Users\alexa\WebstormProjects\RecipeExchangeSystem> npm list -g --depth=0
C:\Users\alexa\AppData\Roaming\JetBrains\WebStorm2024.2\node\versions\20.18.0
+-- corepack@0.29.3
+-- json-server@1.0.0-beta.3
`-- npm@10.8.2

PS C:\Users\alexa\WebstormProjects\RecipeExchangeSystem> 
```

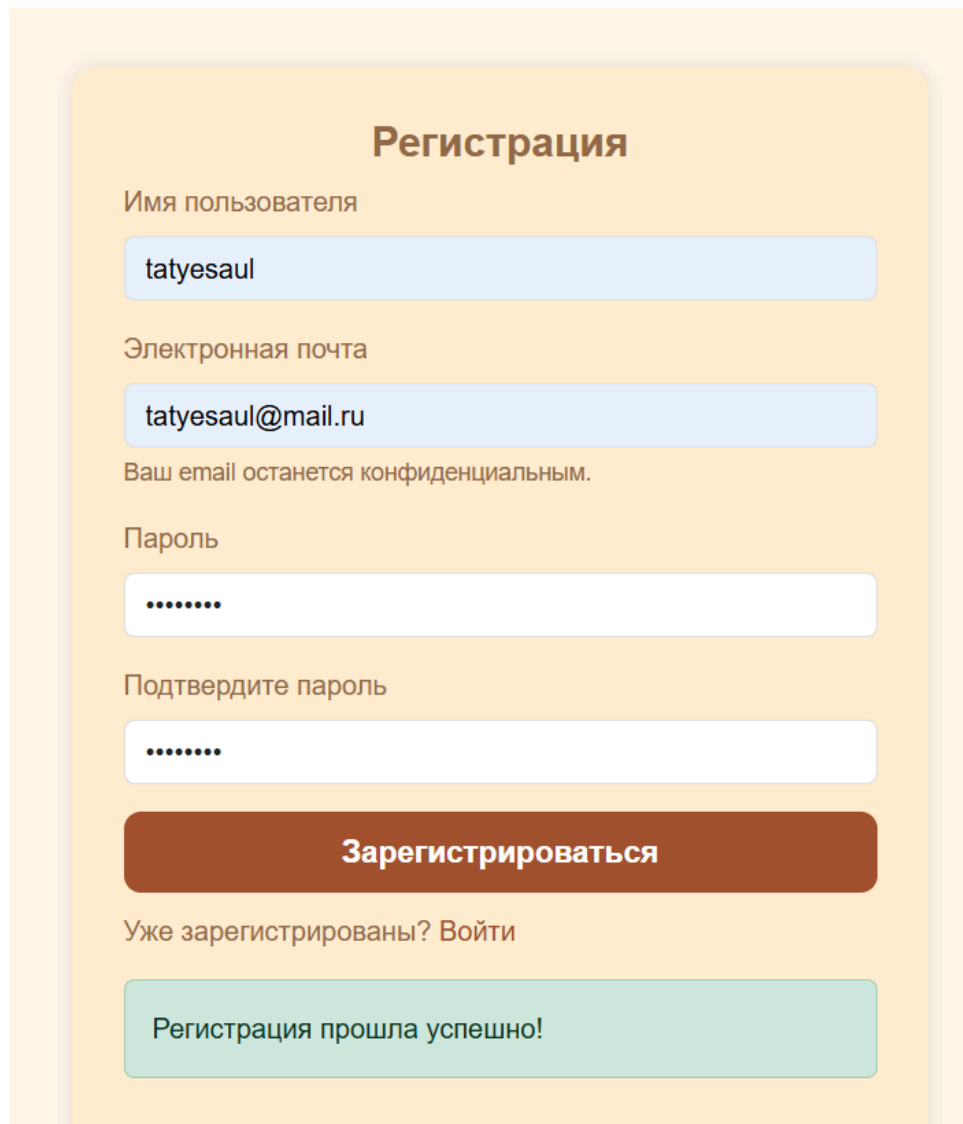
Сервер установился. Создаем файл db.json, выступающий как база данных для мокового апи. Так как по заданию необходимо реализовать авторизацию и регистрацию пользователей на сайте, записываем в файл табличку пользователи, которая будет динамически обновляться при регистрации новых пользователей:

```
{  
  "users": []  
}
```

Запускаем сервер с помощью команды json-server --watch db.json --port 3000:

```
PS C:\Users\alexa\WebstormProjects\RecipeExchangeSystem> json-server --watch db.json --port 3000  
--watch/-w can be omitted, JSON Server 1+ watches for file changes by default  
JSON Server started on PORT :3000  
Press CTRL-C to stop  
Watching db.json...  
  
♡(.-> ♡ <.-),♡  
  
Index:  
http://localhost:3000/  
  
Static files:  
Serving ./public directory if it exists  
  
Endpoints:  
http://localhost:3000/users
```

Сервер запускается и поддерживает эндпоинт /users. Теперь реализуем логику регистрации пользователя в отдельном файле register.js и подключаем его в имеющийся файл верстки.

A screenshot of a web registration form titled "Регистрация" (Registration). The form is set against a light orange background. It contains several input fields: "Имя пользователя" (Username) with the value "tatyesaul", "Электронная почта" (Email) with the value "tatyesaul@mail.ru", and two password fields, both masked with dots. A small text note states "Ваш email останется конфиденциальным." (Your email will remain confidential.). Below the password fields is a large brown button labeled "Зарегистрироваться" (Register). Underneath the button is a link "Уже зарегистрированы? Войти" (Already registered? Log in). At the bottom, a light green box displays the message "Регистрация прошла успешно!" (Registration was successful!).

Регистрация

Имя пользователя

tatyesaul

Электронная почта

tatyesaul@mail.ru

Ваш email останется конфиденциальным.

Пароль

.....

Подтвердите пароль

.....

Зарегистрироваться

Уже зарегистрированы? Войти

Регистрация прошла успешно!

Рисунок 1 – Регистрация нового пользователя

В файле db.json добавился новый пользователь, равно как и на сервере по указанному эндпоинту.

```
"id": "4c93",  
"username": "tatyesaul",  
"email": "tatyesaul@mail.ru",  
"password": "sonechka",  
"registrationDate": "2024-11-22T19:11:07.094Z"  
}
```

Далее подключаем авторизацию – по сути, проверяем, есть ли введенные пользователем данные в базе.

The image shows a user login interface. At the top, the title "Вход в аккаунт" (Login) is centered. Below it, the label "Имя пользователя или e-mail" (Username or e-mail) is followed by a light blue input field containing the text "tatyesaul". Below this, the label "Пароль" (Password) is followed by a white input field with masked characters ".....". A large brown button with the text "Войти" (Login) is positioned below the password field. Underneath the button is the text "Нет аккаунта? Зарегистрироваться" (No account? Register). At the bottom, a light green box contains the message "Вы успешно вошли в аккаунт!" (You have successfully logged in!).

Рисунок 2 – Авторизация пользователя

Теперь реализуем вход в аккаунт – на страницу профиля пользователя загружаем данные с сервера, которые пользователь указывал при регистрации.



Рисунок 3 – Профиль пользователя

В случае, если пользователь не авторизован или же вышел из аккаунта, нажав на соответствующую кнопку в хедере, на странице профиля высвечиваются дефолтные данные, а также всплывает окно “Вы не авторизованы”, перекидывающее на страницу логина:

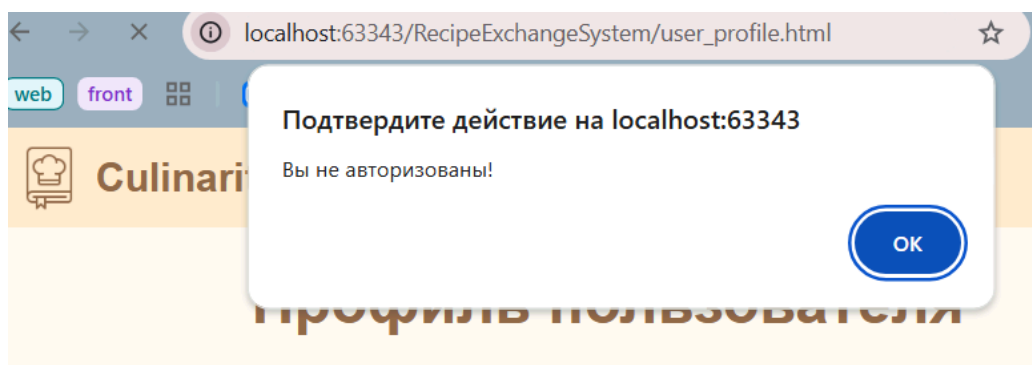
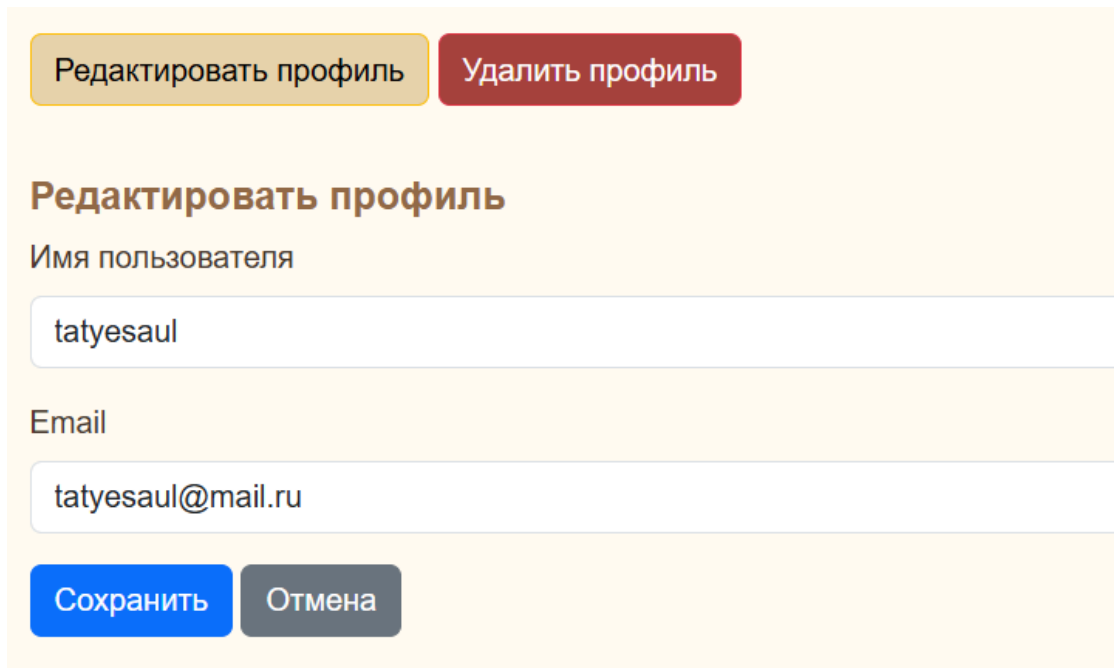


Рисунок 4 – Выход из аккаунта

Если авторизованный пользователь нажимает на кнопку “Редактировать профиль”, у него появляется возможность отредактировать имя пользователя и указанную при регистрации почту. Измененные данные автоматически загружаются на сервер.



Редактировать профиль Удалить профиль

Редактировать профиль

Имя пользователя

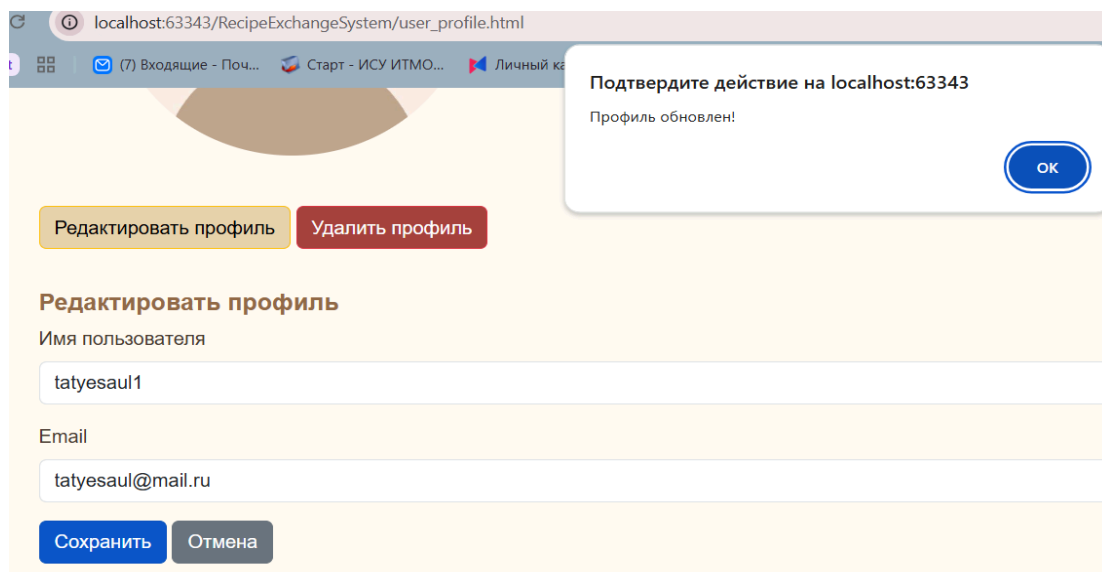
tatyeseul

Email

tatyeseul@mail.ru

Сохранить Отмена

Рисунок 5 – Редактирование профиля



localhost:63343/RecipeExchangeSystem/user_profile.html

(7) Входящие - Поч... Старт - ИСУ ИТМО... Личный к...

Подтвердите действие на localhost:63343
Профиль обновлен!

OK

Редактировать профиль Удалить профиль

Редактировать профиль

Имя пользователя

tatyeseul1

Email

tatyeseul@mail.ru

Сохранить Отмена

Рисунок 6 – Загрузка отредактированных данных

При выборе варианта “Удалить профиль” пользователь удаляется из базы данных.

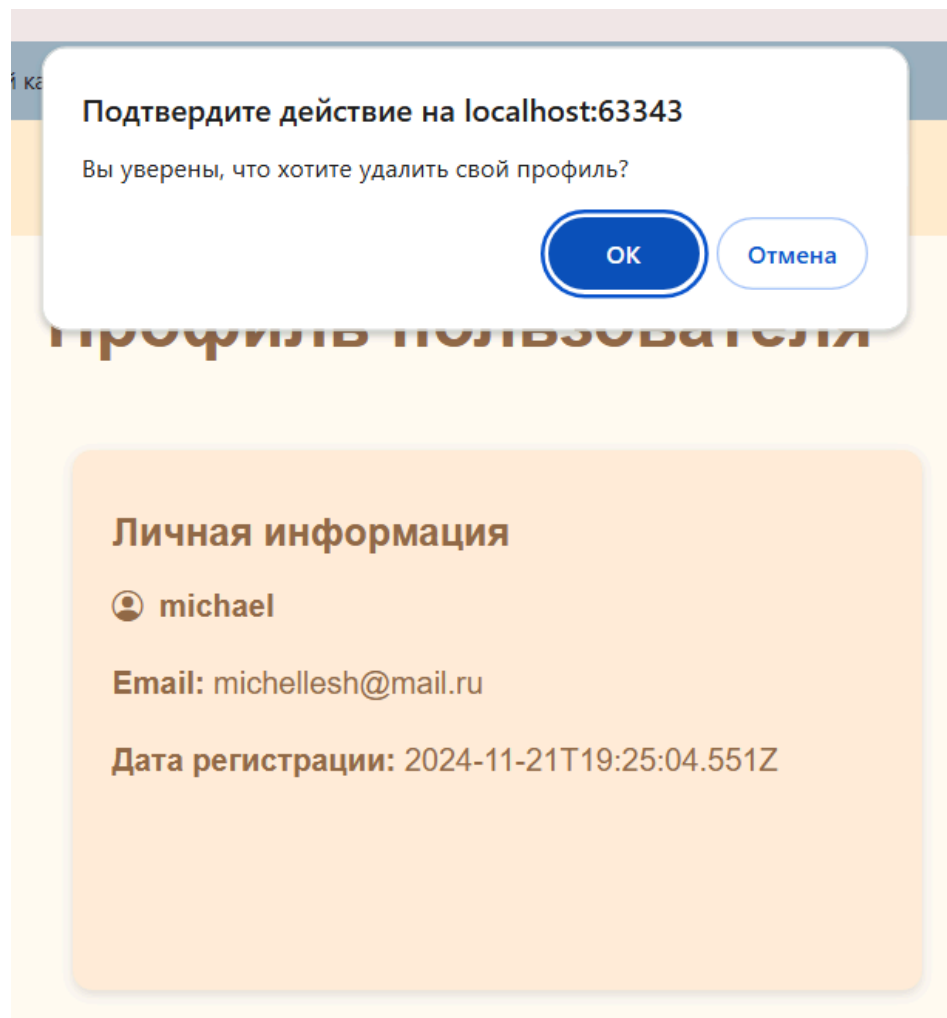


Рисунок 7 – Удаление профиля

Вывод

Было реализовано моковое API средствами JSON-сервера. Так, верстка, выполненная в рамках ЛР 1, была привязана к моковому API с помощью fetch: теперь пользователь может регистрироваться на сайте, авторизовываться и входить в профиль. В профиле пользователь может редактировать данные, внесенные при регистрации, а также удалять профиль с сайта.